DPDP COMPLIANCE AUDIT

saas-copilot

Digital Personal Data Protection Act 2023

31/100

Grade F — Critical

Scanned: 13 April 2026
Files indexed: 44
Languages: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose, uvicorn

CONFIDENTIAL — Internal Use Only

DPDP Compliance Scan Report

Quick Actions for Developers

Fix these first to improve your compliance score:

- **CONSENT_MISSING_REPO_LEVEL** (codebase-wide)
- **CONSENT_WITHDRAWAL_MISSING** (codebase-wide)
- **AUDIT_TRAIL_MISSING** (codebase-wide)
- **NO_DELETION_MECHANISM** → backend/app/api/auth.py
- **DATA_ACCESS_MISSING** (codebase-wide)

This codebase has 8 critical compliance gap(s) that require immediate attention under the Digital Personal Data Protection Act 2023. An additional 3 significant gap(s) were identified that should be addressed within 30 days. The estimated total remediation effort is 3.1–11.4 engineering days.

The application collects user name, email, and password for authentication. The biggest DPDP risk is the potential for unauthorized access to user credentials if the JWT secret is compromised or if there are vulnerabilities in the authentication flow.

Total findings: 17
- Rule Engine: 14
- AI Deep Review: 3

By Severity:
- HIGH: 8
- MEDIUM: 3
- LOW: 1
- INFO: 4
- PASS: 1

Developer Priority:
- Fix Now: 8

Tech Stack: FastAPI, SQLAlchemy, Pydantic, Passlib, python-jose
Risk Profile: The application collects user name, email, and password for authentication. The biggest DPDP risk is the potential for unauthorized access to user credentials if the JWT secret is compromised or if there are vulnerabilities in the authentication flow.

Compliance score by section

DPDP Section	Weight	Score	Status
Section 6	22	0/22 (0%)	FAIL
Section 6(6)	8	0/8 (0%)	FAIL
Section 7	4	4/4 (100%)	PASS
Section 8(1)	18	0/18 (0%)	FAIL
Section 8	10	5.0/10 (50%)	WARN
Section 8(3)	8	4.0/8 (50%)	WARN
Section 8(4)	8	0/8 (0%)	FAIL
Section 8(6)	8	8/8 (100%)	PASS

Section 9	6	6/6 (100%)	PASS
Section 11	6	2.1/6 (35%)	FAIL
Section 16	4	4/4 (100%)	PASS
Section 5	2	0/2 (0%)	FAIL
Section 13 — Grievance Redressal	4	0/4 (0%)	FAIL

Estimated Total Remediation Effort: 3.1–11.4 engineering days

Scan Coverage

Files analyzed: 44

Rule	Section	Severity	File
CONSENT_MISSING_REPO_LEVEL	Section 6 — Consent	HIGH	REPO-WIDE
CONSENT_WITHDRAWAL_MISSING	Section 6(6) — Consent Withdrawal	HIGH	N/A
AUDIT_TRAIL_MISSING	Section 8(4) — Audit Trail	HIGH	N/A
GRIEVANCE_OFFICER_MISSING	Section 13 — Grievance Redressal	HIGH	N/A
PASSWORD_NOT_HASHED	Section 8(1) — Security	HIGH	backend/app/schemas/user.py
NO_DELETION_MECHANISM	Section 8 — Data Principal Rights	MEDIUM	backend/app/api/auth.py
DATA_ACCESS_MISSING	Section 11 — Right to Access	MEDIUM	N/A
RETENTION_MISSING	Section 8(3) — Retention	MEDIUM	backend/app/models/user.py
DATA_PORTABILITY_MISSING	Section 11 — Data Portability	LOW	N/A
NO_THIRD_PARTY_DETECTED	Section 5 — Notice	INFO	N/A
NO_THIRD_PARTY_DATA_SHARING	Section 5 — Purpose Limitation	INFO	N/A
MINIMAL_CONSOLE_LOGGING	Section 8(6) — Breach	INFO	N/A
NO_CROSS_BORDER_DETECTED	Section 16 — Cross-Border Transfer	INFO	N/A
CONSENT_PRESENT	Section 6 — Consent	PASS	backend/app/api/auth.py
DEEP_REVIEW_VALIDATED	Section 5	HIGH	backend/app/db/jwt.py
DEEP_REVIEW_VALIDATED	Section 5	HIGH	backend/app/db/jwt.py
DEEP_REVIEW_VALIDATED	Section 6	HIGH	backend/app/api/auth.py

30-Day Remediation Roadmap

The following plan prioritises findings by severity and estimated effort. Complete Week 1 items before your next security review.

Week 1 — Critical (17–63 hours)

Fix immediately — these represent the highest legal exposure.

- Consent Missing Repo Level — REPO-WIDE (4–16 hrs)
- Consent Withdrawal Missing (1–4 hrs)
- Audit Trail Missing (4–12 hrs)
- Grievance Officer Missing (1–4 hrs)
- Password Not Hashed — schemas/user.py (1–3 hrs)
- Deep Review Validated — db/jwt.py x2 (4–16 hrs)
- Deep Review Validated — api/auth.py (2–8 hrs)

Week 2 — High Priority (6–20 hours)

Address within 2 weeks — short fixes with significant compliance impact.

- No Deletion Mechanism — api/auth.py (2–6 hrs)
- Data Access Missing (2–6 hrs)
- Retention Missing — models/user.py (2–8 hrs)

Week 4 — Advisory (2–8 hours)

Address before next compliance review.

- Data Portability Missing (2–8 hrs)

Total estimated remediation effort: 25–91 hours (3.1–11.4 engineering days)

Details

Finding 1 of 17 — ■ FIX NOW

[Section 6 — Consent]

1. [HIGH] CONSENT_MISSING_REPO_LEVEL — CODEBASE-WIDE:17

Evidence snippet: The application includes routers for authentication and RAG, indicating potential processing of personal data.

Estimated fix time: 4–16 hours

No consent mechanism detected anywhere in the repository.

Risk:

The application collects and processes personal data (name, email, password) without a clear mechanism for users to provide or withdraw consent. This violates DPDP Section 6, potentially leading to unauthorized data processing and loss of user trust.

Fix:

1. Create a consent management endpoint (e.g., POST /consent) to allow users to explicitly consent to data processing, storing this consent status linked to the user's account.

2. Implement a consent withdrawal endpoint (e.g., DELETE /consent) that allows users to revoke their consent, ensuring all associated data processing stops.

3. Modify the signup and login flows to require explicit user consent before processing personal data.

4. Update user profiles to display current consent status and provide a clear link or button to manage consent.

DPDP:

Section 6 — Consent: Personal data shall only be processed on the basis of the consent of the data principal, which shall be freely given, specific, informed, and unambiguous.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.models.user import User from app.schemas.user import UserConsentStatus router = APIRouter(prefix="/consent", tags=["Consent"]) def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.post("/") def grant_consent(user_id: int, db: Session = Depends(get_db)): user = db.query(User).filter(User.id == user_id).first() if not user: raise HTTPException(status_code=404, detail="User not found") user.consent_given = True db.commit() return {"message": "Consent granted successfully"} @router.delete("/") def revoke_consent(user_id: int, db: Session = Depends(get_db)): user = db.query(User).filter(User.id == user_id).first() if not user: raise HTTPException(status_code=404, detail="User not found") user.consent_given = False db.commit() return {"message": "Consent revoked successfully"}
```

Finding 2 of 17 — ■ FIX NOW

[Section 6(6) — Consent Withdrawal]

2. [HIGH] CONSENT_WITHDRAWAL_MISSING — N / A:10

Evidence snippet: The `handleLogout` function is defined, which calls `logout()` from the API services.

Estimated fix time: 1–4 hours

Consent is collected but no withdrawal mechanism detected. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent. Users must be able to revoke consent at any time.

Risk:

While a logout mechanism exists, there is no explicit process for users to withdraw consent for data processing. DPDP Section 6(6) requires consent withdrawal to be as easy as giving consent, and the absence of this feature means users cannot easily revoke their agreement to data processing.

Fix:

1. Implement a dedicated endpoint in the backend (e.g., `POST /consent/withdraw`) that allows users to revoke their consent for data processing, ensuring this action is logged.

2. In the frontend, add a clear and accessible option (e.g., in user settings or a privacy center) for users to trigger the consent withdrawal process.

3. Ensure that upon consent withdrawal, all personal data processing related to that consent is ceased, and any data that is not legally required to be retained is deleted or anonymized.

4. Update the user profile or settings page to reflect the current consent status and provide a direct link to withdraw consent.

DPDP:

Section 6(6) — Consent Withdrawal: A data principal shall have the right to withdraw their consent at any time, and such withdrawal shall be as easy as giving consent.

Example:

```
from fastapi import APIRouter, Depends, HTTPException from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.models.user import User router = APIRouter(prefix="/consent", tags=["Consent"]) def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.post("/withdraw") def withdraw_consent(user_id: int, db: Session = Depends(get_db)): user = db.query(User).filter(User.id == user_id).first() if not user: raise HTTPException(status_code=404, detail="User not found") user.consent_given = False # Add logic here to stop processing related data db.commit() return {"message": "Consent withdrawn successfully"}
```

Finding 3 of 17 — ■ FIX NOW

[Section 8(4) — Audit Trail]

3. [HIGH] AUDIT_TRAIL_MISSING — N / A:17

Evidence snippet: The `log_requests` middleware logs basic request information (method, path, status, duration, trace_id) to standard output.

Estimated fix time: 4–12 hours

No audit trail infrastructure detected. DPDP Section 8(4) requires complete records of personal data processing activities. No access logging, audit model, or change tracking found in the codebase.

Risk:

The application lacks a comprehensive audit trail for personal data processing activities. While basic request logging exists, there's no detailed tracking of data access, modification, or deletion, which is a requirement under DPDP Section 8(4), hindering accountability and incident investigation.

Fix:

1. Create a dedicated `AuditLog` model in SQLAlchemy to store detailed records of data processing events (e.g., user ID, action, timestamp, affected data, IP address).

2. Implement a middleware or decorator that captures relevant events (e.g., user creation, login, data access, data modification) and logs them using the `AuditLog` model.

3. Ensure that sensitive operations, such as password changes or consent modifications, are explicitly logged with sufficient detail.

4. Configure the application to store audit logs securely and retain them according to policy, potentially in a separate, immutable storage.

DPDP:

Section 8(4) — Audit Trail: Complete records of all personal data processing activities shall be maintained.

Example:

```
from sqlalchemy import Column, Integer, String, DateTime, ForeignKey from sqlalchemy.sql
import func from app.db.session import Base class AuditLog(Base): __tablename__ = "audit_logs"
id = Column(Integer, primary_key=True, index=True) user_id = Column(Integer,
ForeignKey("users.id"), nullable=True) action = Column(String, index=True) details =
Column(String) timestamp = Column(DateTime, server_default=func.now()) ip_address =
Column(String) # Example usage in a service function: # from app.db.session import
SessionLocal # db = SessionLocal() # log = AuditLog(user_id=user.id, action="user_login",
details="Successful login", ip_address=request.client.host) # db.add(log) # db.commit()
```

Finding 4 of 17 — ■ FIX NOW

[Section 13 — Grievance Redressal]

4. [HIGH] GRIEVANCE_OFFICER_MISSING — N / A:17

Evidence snippet: The application is a FastAPI backend that includes authentication and RAG functionalities.

Estimated fix time: 1–4 hours

No compliant Grievance Officer / DPO publication found. A PASS requires grievance-related text or a dedicated route **and** a contact email or dedicated grievance/DPO URL. Tiny repos without a grievance route are not marked PASS.

Risk:

The application does not provide any contact information or dedicated channel for users to raise grievances or contact a Data Protection Officer (DPO). This absence violates DPDP Section 13, leaving users without a clear recourse for data protection concerns and potentially leading to unresolved issues.

Fix:

1. Create a new FastAPI endpoint (e.g., `GET /grievance` or `GET /dpo`) that provides contact information for the designated grievance officer or DPO.

2. Add a dedicated contact email address or URL for grievance redressal to the application's configuration or environment variables.

3. In the frontend, add a link to this new endpoint or display the contact information in a visible location, such as the footer or a privacy policy page.

4. Ensure the contact information provided is accurate and monitored for incoming queries.

DPDP:

Section 13 — Grievance Redressal: Data fiduciaries shall establish a mechanism for addressing grievances of data principals regarding the processing of their personal data.

Example:

```
from fastapi import APIRouter import os router = APIRouter(prefix="/support",
tags=["Support"]) @router.get("/grievance") def get_grievance_info(): dpo_email =
os.getenv("DPO_EMAIL", "dpo@example.com") return { "message": "For any data protection
concerns, please contact our Data Protection Officer.", "contact_email": dpo_email,
"instructions": "Please include 'DPDP Grievance' in the subject line." }
```

Finding 5 of 17 — ■ FIX NOW

[Section 8(1) — Security]

5. [HIGH] PASSWORD_NOT_HASHED — backend / app / schemas / user.py:46

Evidence snippet: Calls 'hash_password(password)' to hash the user's password before storing it.

Estimated fix time: 1–3 hours

Model file has password field but no hash/bcrypt/argon/pbkdf2/scrypt/encrypt reference.

Risk:

The application collects user passwords, and while the code shows a `hash_password` function being called, the rule engine flagged this as potentially unhashed. This could lead to unauthorized access to user credentials if the system is breached.

Fix:

1. Verify that the `hash_password` function in `app.core.security` implements a strong, industry-standard hashing algorithm (e.g., bcrypt, Argon2) and that it is correctly applied before storing passwords.

2. Ensure that the `verify_password` function correctly compares the provided password against the stored hash using the same algorithm and salt.

3. Confirm that the `password` field in `backend/app/schemas/user.py` is only used for input and is not directly stored in the database.

4. Review the `password_hash` field in `backend/app/models/user.py` to ensure it is of a type suitable for storing password hashes (e.g., String with sufficient length).

DPDP:

Section 8(1) — Security

Example:

```
from passlib.context import CryptContext # In app/core/security.py crypt =
CryptContext(schemes=["bcrypt"], deprecated="auto") def hash_password(password: str) -> str:
return crypt.hash(password) def verify_password(plain_password, hashed_password) -> bool:
return crypt.verify(plain_password, hashed_password)
```

Finding 6 of 17 — ■ REVIEW & FIX

[Section 8 — Data Principal Rights]

6. [MEDIUM] NO_DELETION_MECHANISM — backend / app / api / auth.py:47

Evidence snippet: Includes the 'auth_router' into the main FastAPI application.

Estimated fix time: 2–6 hours

Deletion-related code detected in models/services but no user-facing deletion endpoint or route handler found. Expose an API or UI for account/data erasure (DPDP Section 8).

Risk:

The application lacks a user-facing mechanism for data deletion, which is a core right for data principals under DPDP Section 8. Without this, users cannot exercise their right to erasure, potentially leading to non-compliance and user distrust.

Fix:

1. Create a new FastAPI router (e.g., `user_data_router`) in `backend/app/api/users.py`.
2. Implement a `DELETE /users/me` endpoint within this new router that requires authentication.
3. In the `DELETE /users/me` endpoint, retrieve the authenticated user's ID and use it to delete their corresponding record from the `users` table in the database.
4. Add the new router to the main application in `backend/app/main.py` using `app.include_router(user_data_router)`.

DPDP:

Section 8 — Data Principal Rights

Example:

```
from fastapi import APIRouter, Depends, HTTPException from sqlalchemy.orm import Session from app.db.session import get_db from app.models.user import User from app.core.auth import get_current_user # Assuming you have this router = APIRouter(prefix="/users", tags=["Users"]) @router.delete("/me") def delete_current_user(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)): user_to_delete = db.query(User).filter(User.id == current_user.id).first() if not user_to_delete: raise HTTPException(status_code=404, detail="User not found") db.delete(user_to_delete) db.commit() return {"message": "User account deleted successfully"}
```

Finding 7 of 17 — ■ REVIEW & FIX

[Section 11 — Right to Access]

7. [MEDIUM] DATA_ACCESS_MISSING — N / A:47

Evidence snippet: Includes the 'auth_router' into the main FastAPI application.

Estimated fix time: 2–6 hours

No endpoint detected that allows authenticated users to retrieve their own personal data. Under DPDP Section 11, Data Principals have the right to access information about their personal data being processed.

Risk:

The application does not provide an endpoint for authenticated users to access their own personal data, violating DPDP Section 11. This prevents data principals from exercising their right to access the information processed about them.

Fix:

1. Create a new FastAPI router (e.g., `user_profile_router`) in `backend/app/api/profile.py`.
2. Implement a `GET /profile` endpoint in this new router that requires authentication.
3. In the `GET /profile` endpoint, retrieve the authenticated user's ID and fetch their details (name, email) from the `users` table.
4. Return the user's personal data in a JSON response, ensuring only necessary fields are exposed.

DPDP:

Section 11 — Right to Access

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.db.session import get_db from app.models.user import User from app.core.auth import get_current_user # Assuming you have this router = APIRouter(prefix="/profile", tags=["Profile"]) @router.get("/") def get_user_profile(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)): return { "id": current_user.id, "name": current_user.name, "email": current_user.email, }
```

Finding 8 of 17 — ■ REVIEW & FIX

[Section 8(3) — Retention]

8. [MEDIUM] RETENTION_MISSING — backend / app / models / user.py:11

Evidence snippet: Defines the 'User' model with fields: id, name, email, password_hash, and created_at.

Estimated fix time: 2–8 hours

Model/schema file contains PII fields but no retention or expiry signals detected. Data may be retained indefinitely.

Risk:

The user model lacks explicit data retention or expiry signals, suggesting PII might be stored indefinitely. This violates DPDP Section 8(3) and increases the risk associated with data breaches or outdated information.

Fix:

1. Add a `deleted_at` or `expires_at` DateTime field to the `User` model in `backend/app/models/user.py`.

2. Implement a background task or a scheduled job (e.g., using APScheduler or Celery Beat) to periodically delete or anonymize user data where `deleted_at` is set or `expires_at` has passed.

3. Modify the user creation logic to set an appropriate `expires_at` date based on a defined retention policy (e.g., 1 year after creation).

4. Implement an endpoint (as per FINDING 2) that allows users to request account deletion, which would set the `deleted_at` field.

DPDP:

Section 8(3) — Retention

Example:

```
from sqlalchemy import Column, Integer, String, DateTime, Boolean from datetime import
datetime, timedelta from app.db.session import Base class User(Base): __tablename__ = "users"
id = Column(Integer, primary_key=True, index=True) name = Column(String(50), nullable=False)
email = Column(String, unique=True, index=True, nullable=False) password_hash = Column(String,
nullable=False) created_at = Column(DateTime, default=datetime.utcnow) expires_at =
Column(DateTime, nullable=True) # New field for retention is_deleted = Column(Boolean,
default=False) # Soft delete flag
```

Finding 9 of 17 — ■ BACKLOG

[Section 11 — Data Portability]

9. [LOW] DATA_PORTABILITY_MISSING — N / A:18

Evidence snippet: The application includes FastAPI, which is a web framework capable of exposing API endpoints.

Estimated fix time: 2–8 hours

No data export or portability endpoint detected. DPDP Section 11 requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Risk:

The application lacks a mechanism to export user data, which is a requirement under DPDP Section 11. Failure to provide this can lead to non-compliance and user dissatisfaction.

Fix:

1. Add a new FastAPI endpoint, e.g., `/users/me/export`, that retrieves the authenticated user's data from the database.

2. Serialize the user data into a JSON format.

3. Return the JSON data with appropriate headers for file download (e.g., `Content-Disposition: attachment; filename="user_data.json"`).

4. Ensure this endpoint is protected and only accessible by the authenticated user.

DPDP:

Section 11(1) — Data Portability requires Data Fiduciaries to provide personal data in a commonly used, machine-readable format upon request.

Example:

```
from fastapi import APIRouter, Depends from sqlalchemy.orm import Session from app.db.session import SessionLocal from app.models.user import User from app.core.security import get_current_user # Assuming you have this utility router = APIRouter(prefix="/users", tags=["Users"]) def get_db(): db = SessionLocal() try: yield db finally: db.close() @router.get("/me/export") def export_my_data(db: Session = Depends(get_db), current_user: User = Depends(get_current_user)): # Fetch user data using current_user.id and db user_data = db.query(User).filter(User.id == current_user.id).first() if not user_data: raise HTTPException(status_code=404, detail="User not found") # Return as JSON, potentially with download headers
```

Finding 10 of 17 — ■ BACKLOG

[Section 5 — Notice]

10. [INFO] NO_THIRD_PARTY_DETECTED — N / A

No known third-party analytics or tracking SDKs detected in codebase.

Finding 11 of 17 — ■ BACKLOG

[Section 5 — Purpose Limitation]

11. [INFO] NO_THIRD_PARTY_DATA_SHARING — N / A

No third-party data sharing detected.

Finding 12 of 17 — ■ BACKLOG

[Section 8(6) — Breach]

12. [INFO] MINIMAL_CONSOLE_LOGGING — N / A

Only minimal client logging (e.g. console.log) detected — not sufficient for breach forensics or server-side audit trails. Add structured application logging and breach alerting (Sentry, PagerDuty, or equivalent).

Finding 13 of 17 — ■ BACKLOG

[Section 16 — Cross-Border Transfer]

13. [INFO] NO_CROSS_BORDER_DETECTED — N / A

No foreign cloud or non-India region config detected.

Finding 14 of 17 — ■ BACKLOG

[Section 6 — Consent]

14. [PASS] CONSENT_PRESENT — backend / app / api / auth.py

Auth enforcement pattern detected inline — consent likely handled at registration.

Finding 15 of 17 — ■ FIX NOW

[Section 5]

15. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / app / db / jwt.py

Estimated fix time: 2–8 hours

The JWT_SECRET is hardcoded as 'dev-secret' when the environment variable is not set.

Finding 16 of 17 — ■ FIX NOW

[Section 5]

16. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / app / db / jwt.py

Estimated fix time: 2–8 hours

The JWT secret key is hardcoded as 'dev-secret' when the JWT_SECRET environment variable is not set. This is a common fallback for development environments and should not be present in production.

Finding 17 of 17 — ■ FIX NOW

[Section 6]

17. [HIGH] DEEP_REVIEW_VALIDATED [AI Deep Review] — backend / app / api / auth.py

Estimated fix time: 2–8 hours

The user ID is included in the JWT payload during signup and login.

AI Deep Compliance Review

This section shows AI analysis that identified compliance gaps beyond deterministic rules. Validated findings are integrated into the main findings list above with [AI Deep Review] labels.

Overall Risk: CRITICAL

The application exhibits critical vulnerabilities in handling user PII, particularly during authentication and data transmission. Sensitive data like passwords are sent in plaintext, and JWT secrets are hardcoded in development configurations, creating a high risk of unauthorized access and data breaches. Storing PII in localStorage without encryption further exacerbates these risks, directly contravening DPDP principles for data protection.

Estimated remediation: 15 engineering day(s)

Most Critical Action: Implement secure transmission of passwords by using HTTPS and ensuring passwords are never sent in plaintext to the backend. This includes encrypting passwords before transmission or using secure authentication protocols.

• **Insecure handling of sensitive credentials and PII** [HIGH] — Data Security and Breach Notification — layers: Data Models & Storage, Authentication & Session, API Routes & Controllers, Third-Party & PII Flows
• **Inadequate data minimization and consent for PII processing** [MEDIUM] — Purpose Limitation and Data Minimization — layers: Data Models & Storage, API Routes & Controllers, Third-Party & PII Flows
• **Insecure storage of authentication tokens and user data** [HIGH] — Data Security and Breach Notification — layers: Data Models & Storage, Third-Party & PII Flows

1. [HIGH] Plaintext password sent to backend during signup — [Third-Party & PII Flows] — Data Security and Breach Notification
The `signup` function sends the user's password in plaintext as part of the JSON request body to the `/auth/signup` endpoint.

2. [HIGH] JWT Secret Key Hardcoded in Dev — [Data Models & Storage] — Data Security and Breach Notification
The JWT_SECRET is hardcoded as 'dev-secret' when the environment variable is not set.

3. [HIGH] PII in JWT without explicit consent — [API Routes & Controllers] — Consent and Lawful Basis for Processing
The user ID is included in the JWT payload during signup and login.

4. [MEDIUM] PII stored in localStorage without encryption — [Third-Party & PII Flows] — Data Security and Breach Notification
The `setToken` function stores the JWT access token in `localStorage`. The `login` and `signup` functions also store the user's name in `localStorage`.

5. [MEDIUM] No PII Encryption in JWT Payload — [Data Models & Storage] — Data Security and Breach Notification
The `create_access_token` function directly encodes a dictionary `data` which may contain PII into a JWT without encryption.

Layer Breakdown
Configuration & Secrets — 1 finding(s). The configuration layer primarily consists of static files and environment variable usage. No direct PII handling is observed, but the configuration of certain parameters could indirectly affect data protection controls.
Data Models & Storage — 4 finding(s). The data models and storage layer shows potential risks related to JWT security, PII handling within tokens, and adherence to data minimization principles in user data collection.
Authentication & Session — 4 finding(s). The authentication layer correctly hashes passwords and uses JWTs for session management. However, there are opportunities to strengthen data minimization in responses and improve the security of JWT secret management and password policies.
API Routes & Controllers — 4 finding(s). The API routes for authentication handle user signup and login. There are potential risks related to PII inclusion in JWTs and data normalization without explicit consent, as well as insufficient error handling for security-critical operations.
Third-Party & PII Flows — 3 finding(s). The application collects PII during signup and login, transmitting passwords in plaintext and storing tokens/user names insecurely in localStorage. While some backend security measures exist, the frontend data handling and PII scope are not fully aligned with DPDP principles.